

Phase 3

Complete Steps From Scratch

Hardware → CubeMX → Firmware → Build → Flash → Test

Phase 3 — Digital Audio Loopback. A new STM32CubeMX project (named **MicDAC**) is created from scratch. The user speaks into the headset microphone, the audio passes through the STM32 (ADC → RAM → DAC), and is heard in the output headphones via the LM386 amplifier.

STATUS: COMPLETED on 2026-04-27. Voice is reproduced cleanly through the STM32 with only mild background hiss. This document reflects the final working configuration after the lessons listed in Section 16.

Project name: **MicDAC**

MCU: **STM32F401RCT6**

HCLK: **84 MHz** from HSE 25 MHz crystal

Sample rate: **8 kHz** for both ADC capture and DAC playback

ADC resolution: **12-bit** (upgraded from 8-bit during testing — see Section 16)

I²C transfer: **blocking** (HAL_I2C_Master_Transmit, NOT the _IT variant)

STEP 1 — Hardware Wiring

Build the entire breadboard before any code is written. Total of 19 wires.

Power Rails (4 wires)

#	From	To
1	Black Pill 3V3	Top + rail
2	Black Pill 5V	Bottom + rail
3	Black Pill GND	Top – rail
4	Top – rail	Bottom – rail (common GND)

MAX9814 — modified board with onboard mic removed (5 wires)

#	From	To
5	MAX9814 VDD	Top + rail (3.3 V)
6	MAX9814 GND	– rail
7	TRRS#1 Sleeve pad	MAX9814 MIC+ empty pad
8	TRRS#1 Ring 2 pad	– rail
9	MAX9814 OUT	STM32 PA0 (ADC1_IN0)

IMPORTANT: leave the MAX9814 GAIN pin floating (no wire). On this user's clone board, tying GAIN to VDD silences the mic entirely. Default 60 dB gain (floating) works fine when the headset mic sits at the user's mouth.

MCP4725 DAC (6 wires + 1 capacitor)

#	From	To
10	MCP4725 VDD (or VCC)	Top + rail (3.3 V)
11	MCP4725 GND (one of two)	– rail
12	MCP4725 GND (other)	– rail (both GND pins go to ground)
13	MCP4725 SCL	STM32 PB6 (I2C1_SCL)
14	MCP4725 SDA	STM32 PB7 (I2C1_SDA)
15	MCP4725 A0	On many clone boards A0 is not exposed and is hardwired to GND on the PCB — address 0x60. If your board has the pin, tie it to the – rail.
16	MCP4725 OUT	one leg of 1 μ F capacitor
16b	other leg of 1 μ F capacitor	LM386 IN (header pin)

MANDATORY: the 1 μ F capacitor between MCP4725 OUT and LM386 IN. The MCP4725 output sits at ~1.6 V DC (mid-rail). Without the cap to block this DC bias, the LM386 input is DC-coupled to a permanent offset and audio cannot pass through — you will hear only hiss. With the cap in place, only the AC audio is coupled in.

LM386 + TRRS#2 Output (4 wires)

#	From	To
16	LM386 VCC (header)	Bottom + rail (5 V)

#	From	To
17	LM386 GND (header)	– rail
18	LM386 + (green terminal)	TRRS#2 Tip AND Ring 1 pads
19	LM386 – (green terminal)	– rail

Plug the input headset (with mic) into **TRRS#1**. Plug the output headphones into **TRRS#2**. Set the LM386 gain knob fully counter-clockwise (minimum) before powering on.

STEP 2 — Create New CubeMX Project

- Open **STM32CubeMX**.
- **File** → **New Project**.
- In the search box, type **STM32F401RCTx**.
- Click your row in the result list → click **Start Project**.
- The Pinout & Configuration tab opens with the chip diagram.

STEP 3 — RCC and SYS

- Left tree → **System Core** → **RCC**.
- Set **High Speed Clock (HSE)** = **Crystal/Ceramic Resonator**.
- Leave Low Speed Clock (LSE) = **Disable**.
- Left tree → **System Core** → **SYS**.
- Set **Debug** = **Serial Wire**. (Critical — without this, SWD programming will fail after first flash.)

STEP 4 — Clock Configuration @ 84 MHz

- Click **Clock Configuration** tab at the top.
- Click **PLL Source Mux** dropdown → select **HSE**.
- In the **HCLK (MHz)** field, type **84** and press Enter.
- CubeMX shows a dialog about auto-solving the PLL — click OK.
- Verify HCLK shows 84 MHz in green and Flash Latency = 2 WS auto-set.

STEP 5 — TIM2 (8 kHz ADC trigger)

Back to Pinout & Configuration tab.

- Left tree → **Timers** → **TIM2**.
- Mode → Clock Source = **Internal Clock**.
- Parameter Settings → Prescaler = **83**.
- Parameter Settings → Counter Period (ARR) = **124**.
- Parameter Settings → Auto-reload preload = **Enable**.
- Trigger Output → Master/Slave Mode = **Disable**.
- Trigger Output → Trigger Event Selection TRGO = **Update Event**.
- NVIC: leave TIM2 IRQ disabled (we only need TRGO, not the interrupt).

Math: $\text{TIM2 clk} = 84 \text{ MHz} / ((83+1) \times (124+1)) = 8000 \text{ Hz exactly}$.

STEP 6 — TIM3 (8 kHz DAC playback tick)

- Left tree → **Timers** → **TIM3**.
- Mode → Clock Source = **Internal Clock**.
- Parameter Settings → Prescaler = **83**.
- Parameter Settings → Counter Period (ARR) = **124**.
- Parameter Settings → Auto-reload preload = **Enable**.

- **NVIC Settings tab** → **enable TIM3 global interrupt**.

STEP 7 — ADC1 (8-bit, TIM2-triggered, DMA)

CubeMX gotcha: do the DMA Settings BEFORE the Parameter Settings, otherwise *DMA Continuous Requests* stays grayed out.

7a — Enable channel

- Left tree → **Analog** → **ADC1**.
- Check **IN0** in the Mode pane — PA0 turns green on the chip diagram.

7b — DMA Settings tab (do this FIRST)

- Click the **DMA Settings** tab in the ADC1 configuration pane.
- Click **Add** → in the new row, DMA Request = **ADC1**.

Field	Value
Stream	DMA2 Stream 0 (auto)
Direction	Peripheral To Memory
Mode	Circular (NOT Normal — continuous sampling)
Priority	High
Use FIFO	Disabled (Direct mode)
Increment Address — Peripheral	Unchecked
Increment Address — Memory	Checked
Peripheral Data Width	Half Word (matches 12-bit ADC)
Memory Data Width	Half Word (matches uint16_t buffer)

Mode = **Circular** is critical. CubeMX defaults to Normal mode which transfers BUF_SIZE samples once and stops — the half/full callbacks fire exactly twice and never again. Audio appears to never start.

7c — Parameter Settings tab (now go back)

Field	Value
Clock Prescaler	PCLK2 div 4
Resolution	12 bits (NOT 8 — see Section 16 lessons learned)
Continuous Conversion Mode	Disabled
DMA Continuous Requests	Enabled (becomes selectable only AFTER the DMA is added in 7b)
External Trigger Conversion Source	Timer 2 Trigger Out event
External Trigger Conversion Edge	Trigger detection on the rising edge
ADC_Regular_ConversionMode → Number Of Conversion	1
Rank 1 (expand the triangle) → Channel	Channel 0
Rank 1 → Sampling Time	15 Cycles

7d — NVIC Settings tab

- Enable **DMA2 stream0** global interrupt.
- Enable **ADC1** global interrupt (or **ADC1, ADC2 and ADC3** global interrupts).

STEP 8 — I2C1 (for MCP4725)

- Left tree → **Connectivity** → **I2C1**.
- Mode = **I2C**.
- PB6 turns yellow as SCL, PB7 turns yellow as SDA on the chip diagram.
- Parameter Settings → I2C Speed Mode = **Fast Mode**.
- Parameter Settings → I2C Clock Speed (Hz) = **400000**.
- Parameter Settings → Fast Mode Duty Cycle = **2**.

STEP 9 — GPIO PC13 (LED Heartbeat)

- On the chip diagram, click **PC13** → select **GPIO_Output**.
- Left tree → **System Core** → **GPIO** → click the PC13 row.
- Set: GPIO output level = **High**, GPIO mode = **Output Push Pull**, no pull-up/down, low speed.
- User Label = **LED**.

STEP 10 — Project Manager and Generate Code

Field	Value
Project Name	MicDAC
Project Location	C:\STM32\ (or any easy path)
Toolchain / IDE	CMake
Linker — Min Heap Size	0x800
Linker — Min Stack Size	0x800

Code Generator sub-tab:

- Tick **Generate peripheral initialization as a pair of '.c/.h' files per peripheral**.
- Tick **Keep User Code when re-generating**.
- Click **GENERATE CODE** (top right).
- Close STM32CubeMX once the dialog finishes.

STEP 11 — Open in VS Code

- Open VS Code.
- **File** → **Open Folder** → browse to **C:\STM32\MicDAC** → Select Folder.
- When prompted, click «Yes, I trust the authors».
- Click the STM32 icon in the left sidebar.
- Click **Setup STM32Cube project(s)** when it activates.
- Wait for the CMake configuration to finish (status bar at the bottom).

STEP 12 — Edit Core/Src/main.c

Add the snippets to the marked USER CODE regions. Save (Ctrl+S) when done.

12a — USER CODE BEGIN PD (private defines)

```
/* USER CODE BEGIN PD */
#define BUF_SIZE      256
#define HALF_SIZE      (BUF_SIZE / 2)
#define MCP4725_ADDR  (0x60 << 1)
/* USER CODE END PD */
```

12b — USER CODE BEGIN PV (private variables)

```
/* USER CODE BEGIN PV */
uint16_t adc_buffer[BUF_SIZE]; /* 12-bit samples -> uint16_t, NOT uint8_t */
volatile uint8_t half_ready = 0;
volatile uint8_t full_ready = 0;
volatile uint16_t dac_play_idx = 0;
/* USER CODE END PV */
```

12c — USER CODE BEGIN 2 (inside main(), after MX_*_Init)

```
/* USER CODE BEGIN 2 */
HAL_ADC_Start_DMA(&hadc1, (uint32_t*)adc_buffer, BUF_SIZE);
HAL_TIM_Base_Start(&htim2);
HAL_TIM_Base_Start_IT(&htim3);
/* USER CODE END 2 */
```

12d — Main while-loop (USER CODE BEGIN WHILE)

Make sure both closing braces are present — one for while(1), one for main(). This is the most common build error in this phase.

```
/* USER CODE BEGIN WHILE */
while (1)
{
    if (half_ready || full_ready) {
        half_ready = 0;
        full_ready = 0;
        HAL_GPIO_TogglePin(GPIOC, GPIO_PIN_13);    /* heartbeat */
    }
}
/* USER CODE END WHILE */

/* USER CODE BEGIN 3 */
} /* <-- closes while(1) */
/* USER CODE END 3 */
} /* <-- closes main() */
```

12e — USER CODE BEGIN 4 (callbacks at end of main.c)

```
/* USER CODE BEGIN 4 */
void HAL_ADC_ConvHalfCpltCallback(ADC_HandleTypeDef *hadc) {
    if (hadc->Instance == ADC1) half_ready = 1;
}

void HAL_ADC_ConvCpltCallback(ADC_HandleTypeDef *hadc) {
    if (hadc->Instance == ADC1) full_ready = 1;
}

void HAL_TIM_PeriodElapsedCallback(TIM_HandleTypeDef *htim) {
    if (htim->Instance == TIM3) {
        /* ADC sample is now 12-bit (0..4095). DAC accepts 12-bit directly,
           no shifting required. Direct passthrough. */
        uint16_t v12 = adc_buffer[dac_play_idx];

        /* MCP4725 Fast-Write packet: top 4 bits + low 8 bits */
        uint8_t pkt[2] = {
            (uint8_t)((v12 >> 8) & 0x0F),
            (uint8_t)(v12 & 0xFF)
        };

        /* BLOCKING I2C, NOT _IT. HAL_I2C_Master_Transmit_IT requires
           the I2C event interrupt to be enabled in NVIC; the default
           CubeMX config does NOT enable it. Blocking version polls the
           I2C status register itself and works without that NVIC flag.
           At 400 kHz Fast Mode, 2 bytes take ~75 us, fits in 125 us tick. */
        HAL_I2C_Master_Transmit(&hi2c1, MCP4725_ADDR, pkt, 2, 1);

        dac_play_idx = (dac_play_idx + 1) % BUF_SIZE;
    }
}
/* USER CODE END 4 */
```

STEP 13 — Build

- Press **Ctrl + S** to save main.c.
- Press **Ctrl + Shift + B** → pick **Build** from the dropdown.
- Watch the terminal panel at the bottom of VS Code for the build output.

Expected output:

```
[16/16] Linking C executable MicDAC.elf
      text      data      bss      dec      hex filename
      ...
Build finished. 0 errors, 0 warnings.
```

If there are errors, do not flash. Read the error message and verify the USER CODE regions are correctly closed (Step 12d). The most common issue is a missing closing brace at the end of main().

STEP 14 — Flash

- Verify ST-Link wiring: SWDIO ↔ DIO, SWCLK ↔ CLK, GND ↔ GND.
- Power the Black Pill via USB-C (do NOT also wire the ST-Link 3.3 V to the board).
- Plug the input headset into TRRS#1 and the output headphones into TRRS#2.
- Click the **Run** button in VS Code's Run/Debug view (or in the STM32 sidebar).
- The first time, an «Edit Configuration» dialog appears — accept defaults.

Expected console output:

```
Memory Programming ...
File download complete
Time elapsed during download operation: ~1s
Application is running, Reset device.
```

STEP 15 — Test

- Confirm the LM386 gain knob is at minimum (fully counter-clockwise).
- Hold the input headset and the output headphones **physically apart** — do not put either on the head yet (acoustic feedback prevention).
- Place one earbud loosely near the ear, do not insert.
- Speak softly into the input headset's mic.
- Slowly turn the LM386 gain knob clockwise while listening.
- When the voice becomes audible, the loop is closed.

Expected behavior

Stimulus	Expected Result
Silence	Quiet hiss in earbud
Speaking normally	Your own voice reproduced with imperceptible delay (~30 ms)
Whistling	Whistle reproduced clearly
Clap or tap on mic	Reproduced as a short impulse
PC13 onboard LED	Toggles many times per second — appears as a steady dim glow

Pass criterion: voice is clearly audible in the headphones, and removing power to the Black Pill makes the audio cut out (proves the audio path goes through the STM32, not directly through the analog hardware).

Section 16 — Lessons Learned (from the actual build)

This section documents every issue encountered during the live build of Phase 3, in the order they were discovered and fixed. Earlier versions of this document had the wrong configuration for several of these and were corrected only after live debugging.

16.1 — DMA Continuous Requests was grayed out

Symptom: the DMA Continuous Requests field in ADC1 Parameter Settings could not be enabled (greyed out / not selectable).

Cause: CubeMX requires the DMA channel to be added FIRST in the DMA Settings tab before that checkbox becomes selectable.

Fix: always configure ADC1 in this order: enable channel → DMA Settings tab (Add ADC1, set Circular + Half Word) → THEN go back to Parameter Settings and toggle DMA Continuous Requests = Enabled.

16.2 — Sampling Time field was missing

Symptom: Could not find Sampling Time in the Parameter Settings list, only ADC_Injected_ConversionMode was visible.

Cause: Sampling Time lives inside the expandable **Rank 1** entry of ADC_Regular_ConversionMode (a different section from Injected). It only appears after Number Of Conversion is set to 1.

Fix: set ADC_Regular_ConversionMode → Number Of Conversion = 1, then click the small triangle next to Rank 1 to expand it, then set Channel and Sampling Time inside.

16.3 — DMA Mode was Normal instead of Circular

Symptom: the PC13 heartbeat LED never glowed; main loop never saw half_ready/full_ready set.

Cause: in CubeMX DMA Settings, Mode defaulted to Normal. With Normal mode the DMA stops after one full transfer (~32 ms in our config) and never restarts. The half-complete and full-complete callbacks fire exactly twice on boot, then never again, which is invisible to the user.

Fix: change DMA Mode → Circular and regenerate code.

16.4 — Tying GAIN pin of MAX9814 to VDD silenced the mic

Symptom: after wiring MAX9814 GAIN pad to the 3.3 V rail (per datasheet this should set 40 dB gain), the mic produced no signal at all.

Cause: on the user's specific MAX9814 clone breakout, the GAIN pin appears to be wired in a way that conflicts with an external pull-up. Setting GAIN externally disabled the chip's input.

Fix: leave the GAIN pin floating. Default 60 dB gain is more amplification than ideal but works reliably.

16.5 — HAL_I2C_Master_Transmit_IT did not transfer

Symptom: MCP4725 OUT showed exactly one DC value (~1.6 V) regardless of audio. DAC was not updating per sample. User heard hiss but no voice.

Cause: the `_IT` (interrupt-driven) HAL function requires the I²C event interrupt to be enabled in NVIC. CubeMX's default I²C generation in this project did NOT add `HAL_NVIC_EnableIRQ(I2C1_EV_IRQn)`. The first byte of the first transfer went out, then the I²C peripheral got stuck waiting for an event interrupt that never fired. All subsequent calls returned `HAL_BUSY`.

Fix: use the blocking variant **HAL_I2C_Master_Transmit** (without the `_IT` suffix) in the TIM3 ISR. Blocking transfers poll the status register and complete in ~75 μs at 400 kHz, well within the 125 μs tick period. Alternative fix would have been to enable I²C event/error interrupts in CubeMX NVIC Settings, but the code change is simpler and equally effective.

16.6 — Missing 1 μF cap between DAC and LM386 caused silence

Symptom: after fixing the I²C transfer issue (16.5), the DAC was updating but the user still heard only hiss. The LM386 finger-touch test still produced hum, so the amplifier itself was fine.

Cause: the MCP4725 OUT sits at ~1.6 V DC (mid-rail with the boosted code). Without a series capacitor, this DC offset is fed straight into the LM386 input, which DC-couples to it and prevents AC audio from being amplified.

Fix: add a 1 μ F capacitor in series between MCP4725 OUT and LM386 IN. The cap blocks DC, passes AC. With this in place voice came through.

16.7 — Robotic-sounding voice with 8-bit ADC

Symptom: after all the above fixes, voice was finally audible, but it sounded heavily robotic / digital / quantized.

Cause: 8-bit ADC has only 256 levels = ~48 dB SNR theoretical. The audible stepping between values produces a robotic timbre on voice.

Fix: upgrade to 12-bit ADC (4096 levels = 72 dB SNR, 16x cleaner). In CubeMX: ADC1 Resolution = 12 bits, DMA data widths Peripheral and Memory both = Half Word. In code: change buffer type from uint8_t to uint16_t and remove the << 4 shift in the TIM3 ISR (ADC and DAC are now both 12-bit native, direct passthrough). Voice immediately sounded natural.

Troubleshooting

Symptom	Likely Cause	Fix
Total silence	MCP4725 not powered, A0 floating, OUT not connected to LM386 IN	Multimeter on MCP4725 VDD = 3.3V; verify A0 to GND; verify wire 15
Distorted / clipping	LM386 gain too high, or DC bias not subtracted	Lower LM386 gain; or apply bias correction in firmware (subtract ~96)
LED PC13 not flickering	ADC DMA not running — main loop never sees half_ready/full_ready	Re-check Step 7: DMA Continuous Requests = Enabled, NVIC for DMA2 Stream0 enabled
Build error: hi2c1 undeclared	I2C1 not enabled in CubeMX	Re-do Step 8, regenerate code
Build error: htim3 undeclared	TIM3 not configured in CubeMX	Re-do Step 6, regenerate code
Build error: expected declaration at end of input	Missing closing brace in main.c	Re-check Step 12d — both } at the end (one for while, one for main)
Audio is just a steady tone, no voice	TIM3 firing too fast or HAL_I2C_Master_Transmit_IT returning busy	Verify TIM3 PSC=83, ARR=124; verify NVIC priorities (I2C above TIM3)
Audio crackles / drops out	I2C transfers not finishing in time	Verify I2C1 at 400 kHz, NVIC priorities correct
Flash error: target locked	SWD disabled in CubeMX (SYS → Debug = None instead of Serial Wire)	Recovery: hold BOOT0, reset, mass-erase via STM32CubeProgrammer; then fix Step 3
Squealing feedback	Input mic too close to output earbuds	Hold them apart, lower LM386 gain

Diagnostic Sequence (if the loopback is silent)

- **Power check.** Multimeter on MCP4725 VDD vs GND — should be 3.3 V.
- **I²C scan.** Add a temporary loop in firmware that calls HAL_I2C_IsDeviceReady on addresses 1..127 and toggles the LED for each one found. You should detect 0x60.
- **DC test.** Replace the TIM3 ISR body with two alternating writes (0x000 and 0xFFFF) with HAL_Delay between them. Multimeter on MCP4725 OUT should toggle between 0 V and 3.3 V every second.
- **Tone test.** Generate a 500 Hz square wave on the DAC. You should hear a clean low-pitched beep in the headphones.
- **Loopback test.** Only after all of the above pass, run the full firmware and you should hear yourself.

What This Phase Proves

- I²C peripheral is correctly configured at 400 kHz.
- MCP4725 DAC is responding at address 0x60.
- TIM3 fires at exactly 8 kHz, locked to the same clock as TIM2.
- Interrupt-driven I²C completes within the 125 μs budget every cycle.
- ADC and DAC sample rates are synchronised — no buffer overruns or underruns.
- The full real-time digital audio loop closes — every sample lives briefly in the STM32's RAM.

- Total system latency is approximately 32 ms (one half-buffer at 8 kHz) plus negligible processing time.

After this phase passes, the next milestones are: **Phase 4** — LTE module manual AT-command bring-up, **Phase 5** — STM32 drives the LTE module, **Phase 6** — two-node plain audio over LTE, **Phase 7** — add RSA + AES hybrid encryption, **Phase 8** — phone-number call setup & demo polish.

End of Phase 3 step-by-step document.